

HMDP Production Hardening Checklist

Scope

- Login
- Seckill
- Cache
- Testing
- Deployment
- Monitoring

Priority Guide

- P0: Must fix first. Directly affects correctness, security, or core interview quality.
- P1: Should fix. Strongly improves engineering quality and production credibility.
- P2: Nice to have. Good enhancement after core gaps are closed.

1. Login

P0

- Add RBAC / authorities support.
Current authentication works, but authorities are still effectively empty.
- Add user status checks.
Block disabled, deleted, or frozen users during login and refresh.
- Externalize all security-sensitive config.
JWT secret, cookie flags, mail config, Redis/DB endpoints must be environment-driven.
- Clarify Access Token / Refresh Token invalidation behavior.
Define what happens to old access tokens after refresh rotation.

P1

- Add Refresh Token reuse detection.
If an old refresh token is used again, treat it as a security event.
- Complete cookie security settings.
Secure, SameSite, environment-specific settings.
- Add IP-level rate limiting in addition to current email-based rate limiting.
- Add login / refresh / logout audit logs.

P2

- Add device/session management.
- Add suspicious login detection and account lock strategy tuning.

2. Seckill

P0

- Add seckill start/end time validation in the real purchase path.
- Unify user context source.
Remove mismatch risk between UserHolder and SecurityContextHolder.
- Add database unique constraint on (voucher_id, user_id).
Current count check is business-level protection, not the strongest DB guarantee.
- Complete failure compensation after Redis pre-deduction succeeds but DB finalization fails.
Include stock rollback and user qualification rollback.

P1

- Add retry limit for message consumption.
- Add dead-letter handling for poison messages.

- Add idempotent consumption design.
 - Add better consumer failure logging and alerting.
 - Clarify scaling plan for consumer model.
- Current single-thread consumer is acceptable for learning, but not enough for production discussion.

P2

- Add stronger anti-abuse controls.
IP throttling, user throttling, optional anti-bot or pre-token mechanism.
- Add post-order event publishing strategy if Kafka is introduced later.

3. Cache

P0

- Actually connect cache breakdown strategies to business query path.
queryById currently uses only queryWithPassThrough().
- Add TTL randomization to reduce avalanche risk.
- Fix ShopType cache strategy.
Add TTL and explicit invalidation/update policy.
- Upgrade lock implementation safety.
Current tryLock/unlock is basic and not production-grade.

P1

- Add cache hit/miss and rebuild visibility.
- Standardize cache key naming and TTL conventions.
- Document when to choose:
queryWithPassThrough()
queryWithLogicalExpire()
queryWithMutex()

P2

- Add Bloom filter discussion or implementation if future scale justifies it.
- Consider local cache + Redis multi-level cache only after core path is stable.

4. Testing

Current State

- Only a minimal SpringBootTest shell exists.
- No meaningful automated tests yet.

P0

- Add targeted tests for Login.
Code rate limiting, code retry limits, refresh flow, logout invalidation.
- Add targeted tests for Seckill.
Lua result branches, duplicate order protection, pending-list compensation, DB fallback path.
- Add targeted tests for CacheClient.
Pass-through, logical-expire, mutex rebuild, empty-cache behavior.

P1

- Split tests by level:
unit tests
integration tests
- Add abnormal-path tests:
Redis miss
DB empty
duplicate order
failed consumer retry

P2

- Add test data management and reusable fixtures.
- Add lightweight performance or concurrency verification for critical paths.

5. Deployment

Current State

- README is nearly empty.
- No Dockerfile or docker-compose found.
- application.yaml still contains hardcoded infra addresses and credentials.

P0

- Add Dockerfile.
- Add docker-compose for app + MySQL + Redis.
- Remove hardcoded infra config from application.yaml.
Use environment variables or profile-specific config.
- Write a real README.
Include architecture, startup steps, required services, module summary.

P1

- Add dev / test / prod config separation.
- Add DB migration tooling plan.
Flyway or Liquibase.

P2

- Add CI/CD basics.
- Add deployment notes for cloud or VM target environment.

6. Monitoring

Current State

- No actuator / metrics / dashboards found.
- No structured production monitoring setup found.

P0

- Add structured logs for key flows.
Login, refresh, logout, seckill success/failure, compensation.
- Keep sensitive-data masking consistent.
- Add business-level metrics:
login failures
code send frequency
seckill success/failure
pending-list backlog
cache hit/miss

P1

- Add health checks and metrics exposure.
- Add basic alert design:
repeated consumer failures
dead-letter growth (if introduced)
Redis or DB unavailability

P2

- Add Prometheus + Grafana.
 - Add tracing only after metrics/logging baseline is in place.
-

Recommended Execution Order

Phase 1

- Login P0
- Seckill P0
- Cache P0

Phase 2

- Testing P0/P1
- Deployment P0

Phase 3

- Monitoring P0/P1

Phase 4

- Optional Payment module

Only after the current three core modules are strengthened and testable.

Top 10 Highest-Value Upgrades

1. Add RBAC / authorities to Login.
2. Add user status validation to Login.
3. Add seckill time validation.
4. Add DB unique constraint for seckill orders.
5. Add seckill compensation closure for Redis/DB inconsistency.
6. Connect cache breakdown strategy to real business query path.
7. Add TTL randomization for cache keys.
8. Add real automated tests for the three core modules.
9. Add Docker + docker-compose.
10. Rewrite README and externalize environment config.

Outcome Goal

After these upgrades, the project should move from:

- strong junior / junior-mid signal

Toward:

- junior-mid with clear mid-level engineering signals

The project will become stronger not because it has more features, but because the existing core modules become more correct, more testable, more deployable, and more production-credible.